Project Executable Files

Date	08/07 2026
Team ID	
Project Name	A comprehensive measure of Well Being
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for the HDI Predictor — Policy Reviewer project. All files are organized, named, and uploaded to the specified submission platform. The evaluator can run or deploy the solution locally or access the live hosted version using the instructions provided below.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	package.json (Node.js / npm dependency file)	Yes
4	Database schema / seed files (if applicable)	NA — static in-memory datasets (data.ts), no external DB
5	Environment configuration file (.env.example or similar)	Yes — GEMINI_API_KEY required in .env.local
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA — web-based application only
8	Dockerfile / Containerization config (if applicable)	No — not yet containerized; planned enhancement
9	Test files and test results	Yes — performance test results included
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Overview of the HDI Predictor project's file and folder structure:

```
/hdi-predictor-policy-reviewer
/src - React + TypeScript source code App.tsx -
Main application logic and state management
main.tsx - React application entry point
types.ts - TypeScript interfaces (Preset, Policy, CountryData, HdiComputation)
data.ts - Static datasets (SCENARIOS, POLICY_INTERVENTIONS, COUNTRIES,
GLOBAL_AVERAGES) index.css - Global stylesheet /components
HdiPredictor.tsx - Interactive slider-based HDI predictor UI
GlobalComparison.tsx - Country benchmarking dashboard
HdiReport.tsx - HDI summary report generator
AboutHdi.tsx - HDI methodology explainer page
AboutProject.tsx - Project information page
/assets - Static images and icons
server.ts - Express backend (POST /api/advisor - Gemini AI
integration) index.html - Application HTML entry point package.json -
npm dependencies and scripts vite.config.ts - Vite build
configuration tsconfig.json - TypeScript compiler configuration
metadata.json - Project metadata
README.md - Setup and run instructions
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://hdi-predictor.vercel.app
Login Credentials (Demo)	Not required — the tool is publicly accessible with no authentication
Platform / Hosting Provider	Vercel (frontend, static hosting) + Node.js Express server for the /api/advisor endpoint
Repository Link	https://github.com/<team>/hdi-predictor-policy-reviewer
Demo Video Link	https://drive.google.com/<demo-walkthrough-link>

Step 4: Run Instructions

Step-by-step instructions for the evaluator to run the project locally:

Pre-requisites:
Node.js v18+ and npm installed. A valid Gemini API key (for the AI Policy Advisor feature).

Step 1: Clone the repository: git clone https://github.com/<team>/hdi-predictor-policy-reviewer
Step 2: Install dependencies: npm install

Step 3: Configure environment: create .env.local and set GEMINI_API_KEY=<your-api-key>
Step 4: Run the application: npm run dev
Step 5: Open browser at http://localhost:5173
Step 6 (optional): To test the AI Advisor backend independently, run: npm run server

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	AI Policy Advisor response time depends on external Gemini API latency (avg 1.4s, peak 3.2s)	Debounced trigger + retry with exponential backoff implemented; acceptable for current use case
2	No persistent database — country and scenario data is static and in-memory only	Sufficient for current scope; database integration planned for multi-user session history in future release
3	Not yet containerized (no Dockerfile)	Planned enhancement for standardized deployment across environments